CS 645

Security and Privacy in Computer Systems

(Spring 2024)

Fuad Hamidli

Department of Computer Science

NJIT New Jersey's Science & Technology University

CS 645

Security and Privacy in Computer Systems

Lecture 1

- Course overview
- Introduction
- Crypto crash course

Course information

- When and where?

  Wednesday 6:00 – 8:50pm @ Jersey City campus
- CS 645 (1j2)
- Course webpage (general information, syllabus, course slides, projects, etc.):

  https://canvas.njit.edu/
- Class attendance is strongly recommended
- If you miss a lecture, it is your responsibility to find out what happened in class

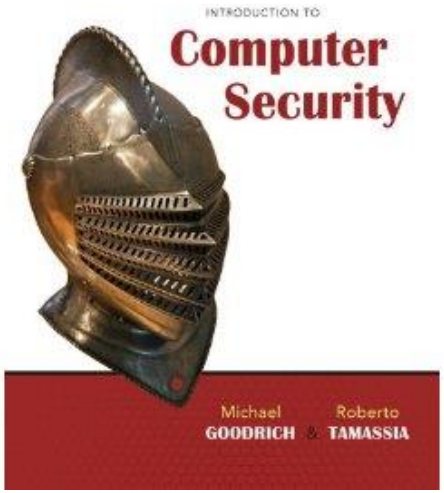
Course information

- Lecturer contact info:
  - Email: fuad.hamidli@njit.edu
  - Office hours:
    - Tuesday 4:00 – 5:00 pm @Newark GITC 4202
    - Wednesday 5:00-6:00 pm @Jersey city

    - Also, by appointment via email

Textbook

- *"Introduction to Computer Security"*
  by M. Goodrich and R. Tamassia,
  Addison Wesley, 2010, ISBN: 032151294



- In addition, course material will include research articles from electronic databases such as:
  - ACM Digital Library: http://dl.acm.org
  - IEEE Xplore: http://ieeexplore.ieee.org
  - Science Direct: http://www.sciencedirect.com

Grading policy

- 3 mini-projects 45%
- Midterm Exam 25%
- Final Exam 30%

Up to 10% extra credit for active participation in class

No make up exams

Mini-projects

- Mini-projects
  - must be **typed**
  - are due in the beginning of the class
    - If you cannot attend class, assignment should be sent by email 1 hour before class starts

Academic integrity

- The NJIT University code on academic integrity will be followed:

  https://www.njit.edu/policies/sites/policies/files/academic-integrity-code.pdf

(Some excerpts): You should not:

- Take an examination for another student.
- Plagiarize, in part, written, oral or graphic work which was authored or prepared by another.
- Fail to acknowledge that the work submitted for credit is the work of a collaboration.
- Cheat on an examination.

Note in particular that cheating on exams, copying homework assignments and exam papers, and plagiarizing (in full or in part) someone else's work is forbidden. Students will be sent to the Dean of Students when such situations arise, for disciplinary actions.

Academic Integrity

- *Please note that it is my professional obligation and responsibility to report any academic misconduct to the Dean of Students Office. Any student found in violation of the code by cheating, plagiarizing or using any online software inappropriately will result in disciplinary action. This may include a failing grade of F, and/or suspension or dismissal from the university.*

- *As part of projects, students must turn in code or work that has fully been written by him/her and no-one else. Any submitted text or code (even few lines) obtained through the Internet or otherwise, or is product of someone else's work, risks severe punishment, as outlined by the University; all parties of such interaction receive automatically 0 and grade is lowered by one or two levels.*

Important dates

- Last day to withdraw: Apr 1
- Reading days: May 1, May 2
- Final exam: May 8 (to be confirmed)

NJIT Spring 2024 academic calendar:

https://www.njit.edu/registrar/spring-2024-academic-calendar

Important dates

2/14/24	Project 1 posted
2/28/24	Project 1 due
3/6/24	Midterm exam
3/27/24	Project 2 posted
4/10/24	Project 2 due
4/10/24	Project 3 posted
4/24/24	Project 3 due
5/8/24	Final exam (to be confirmed)

(project dates are tentative and might change)

Other important stuff

- You are advised to take notes

- Prerequisites
  - There are no specific course prerequisites for the course, but students are expected to enter this course with a basic knowledge of operating systems, networking, algorithms, and data structures.
  - Also, students should be able to program in Java and C/C++.

Other important stuff

- When you send me an email about this class, please start the subject line with “CS.645”

Questions?

Course outline

- Introduction (overview of course topics, security goals, overview of attacks)
- Crypto crash course
- Access control mechanisms
- Operating systems security
- Software security, Secure Programming
- Malicious code, Malware, Rootkits
- Web security
- Trusted computing
- Introduction to security of networked systems
- Privacy and anonymity on the Web
- Content protection, Software obfuscation, Digital rights management
- Database security
- Security of electronic voting
- Computer crime - laws and ethics, Security & privacy policy (Sarbanes Oxley, HIPAA)
- Miscellaneous topics: side-channel attacks, gaming security, information assurance (common criteria), risk analysis

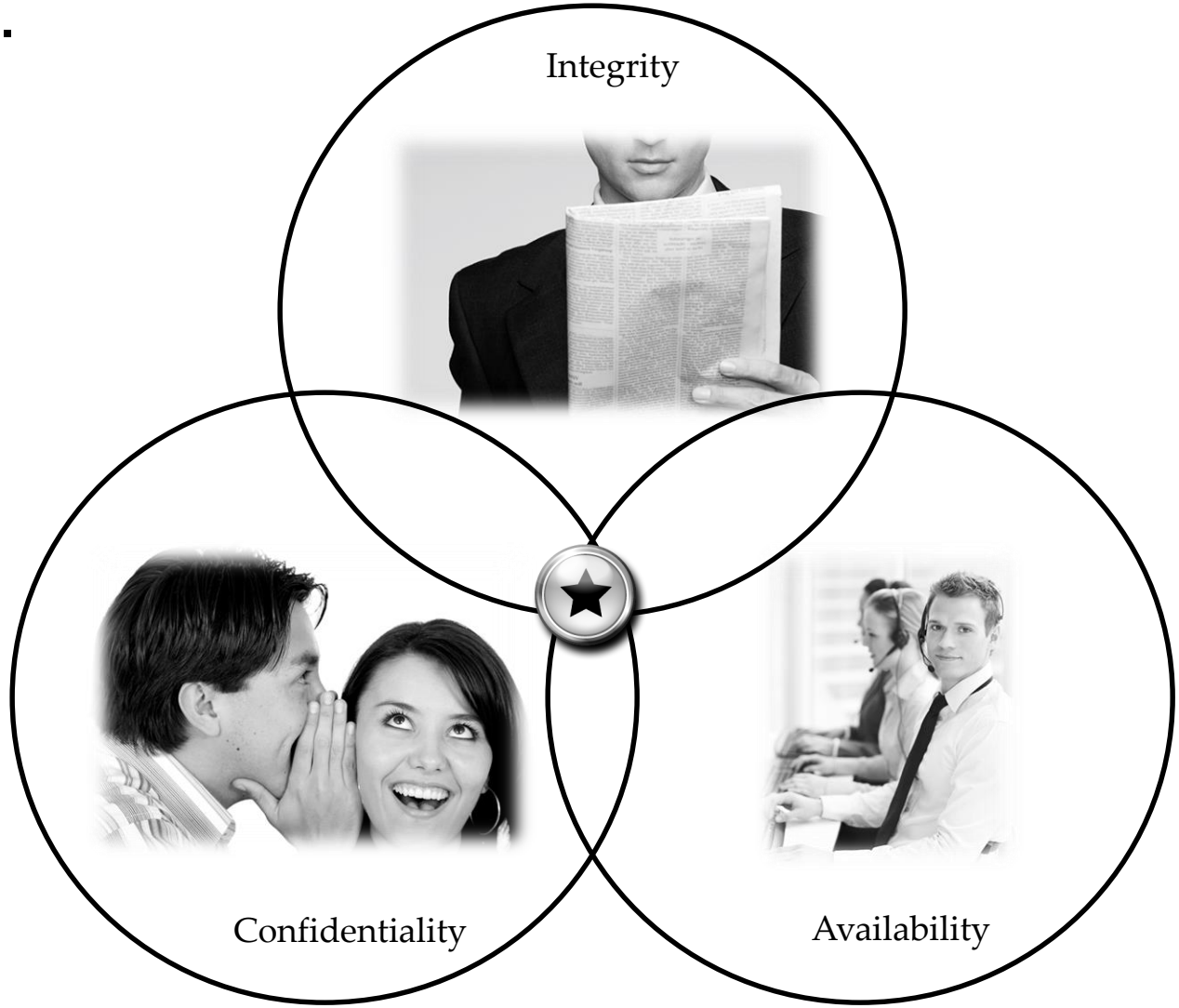
Defining Security

- The security of a system, application, or protocol is always relative to:
  - A set of desired properties
  - An adversary with specific capabilities

- For example, standard file access permissions in Linux and Windows are not effective against an adversary who can boot from a CD

Security Goals

- C.I.A.

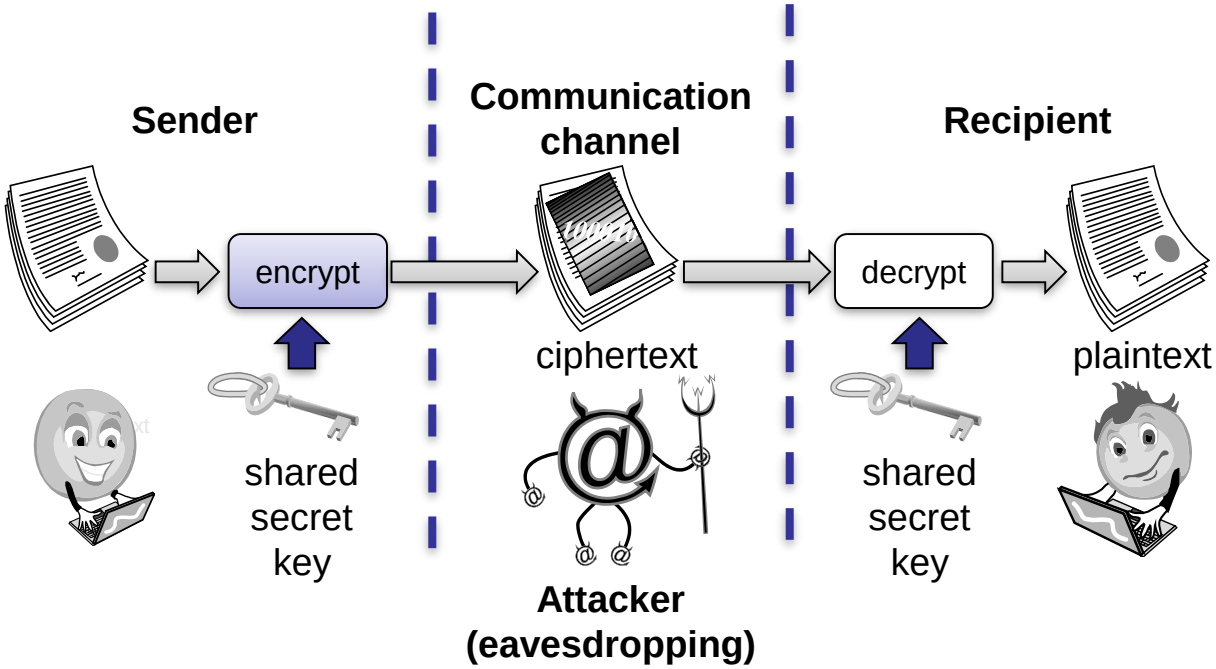


Confidentiality

- **Confidentiality** is the avoidance of the unauthorized disclosure of information.
  - confidentiality involves the protection of data, providing access for those who are allowed to see it while disallowing others from learning anything about its content.

Tools for Confidentiality

- **Encryption:** the transformation of information using a secret, called an encryption key, so that the transformed information can only be read using another secret, called the decryption key (which may, in some cases, be the same as the encryption key).

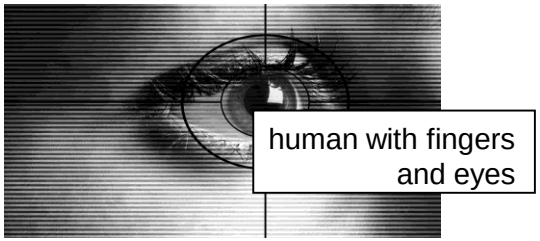


Tools for Confidentiality

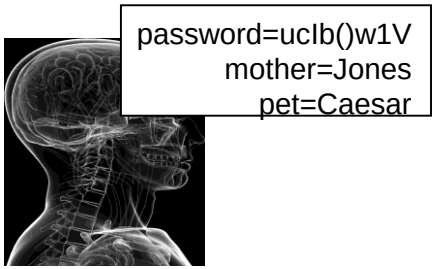
- **Access control:** rules and policies that limit access to confidential information to those people and/or systems with a "need to know."
  - This need to know may be determined by identity, such as a person's name or a computer's serial number, or by a role that a person has, such as being a manager or a computer security specialist.

Tools for Confidentiality

- **Authentication:** the determination of the identity or role that someone has. This determination can be done in a number of different ways, but it is usually based on a combination of
  - something the person has (like a smart card or a radio keychain storing secret keys),
  - something the person knows (like a password),
  - something the person is (like a human with a fingerprint).



Something you are



Something you know

Something you have

Tools for Confidentiality

- **Authorization:** the determination if a person or system is allowed access to resources, based on an access control policy.
  - Such authorizations should prevent an attacker from tricking the system into letting him have access to protected resources.
- **Physical security:** the establishment of physical barriers to limit access to protected computational resources.
  - locks on cabinets and doors
  - the placement of computers in windowless rooms
  - the use of sound dampening materials
  - the construction of buildings or rooms with walls incorporating copper meshes (called **Faraday cages)** so that electromagnetic signals cannot enter or exit the enclosure.

Integrity

- **Integrity:** the property that information has not be altered in an unauthorized way.
- **Tools:**
  - **Backups:** the periodic archiving of data.
  - **Checksums:** the computation of a function that maps the contents of a file to a numerical value. A checksum function depends on the entire contents of a file and is designed in a way that even a small change to the input file (such as flipping a single bit) is highly likely to result in a different output value.
  - **Data correcting codes:** methods for storing data in such a way that small changes can be easily detected and automatically corrected.
  - **Cryptographic "checksums"** (message authentication codes) – can ensure protection against malicious alteration
    - Regular checksums provide protection only against accidental alteration

Availability

- **Availability:** the property that information is accessible and modifiable in a timely fashion by those authorized to do so.
- **Tools:**
  - **Physical protections:** infrastructure meant to keep information available even in the event of physical challenges.
  - **Computational redundancies:** computers and storage devices that serve as fallbacks in the case of failures.

Other Security Concepts

- A.A.A.

Authenticity

Assurance

Anonymity

Assurance

- **Assurance** refers to how trust is provided and managed in computer systems.
- **Trust management** depends on:
  - **Policies,** which specify behavioral expectations that people or systems have for themselves and others.
    - For example, the designers of an online music system may specify policies that describe how users can access and copy songs.
  - **Permissions,** which describe the behaviors that are allowed by the agents that interact with a person or system.
    - For instance, an online music store may provide permissions for limited access and copying to people who have purchased certain songs.
  - **Protections,** which describe mechanisms put in place to enforce permissions and polices.
    - We could imagine that an online music store would build in protections to prevent people from unauthorized access and copying of its songs.

Authenticity

- **Authenticity** is the ability to determine that statements, policies, and permissions issued by persons or systems are genuine.
- **Primary tool:**
  - **digital signatures.** These are cryptographic computations that allow a person or system to commit to the authenticity of their documents in a unique way that achieves **nonrepudiation** (the property that authentic statements issued by some person or system cannot be denied).

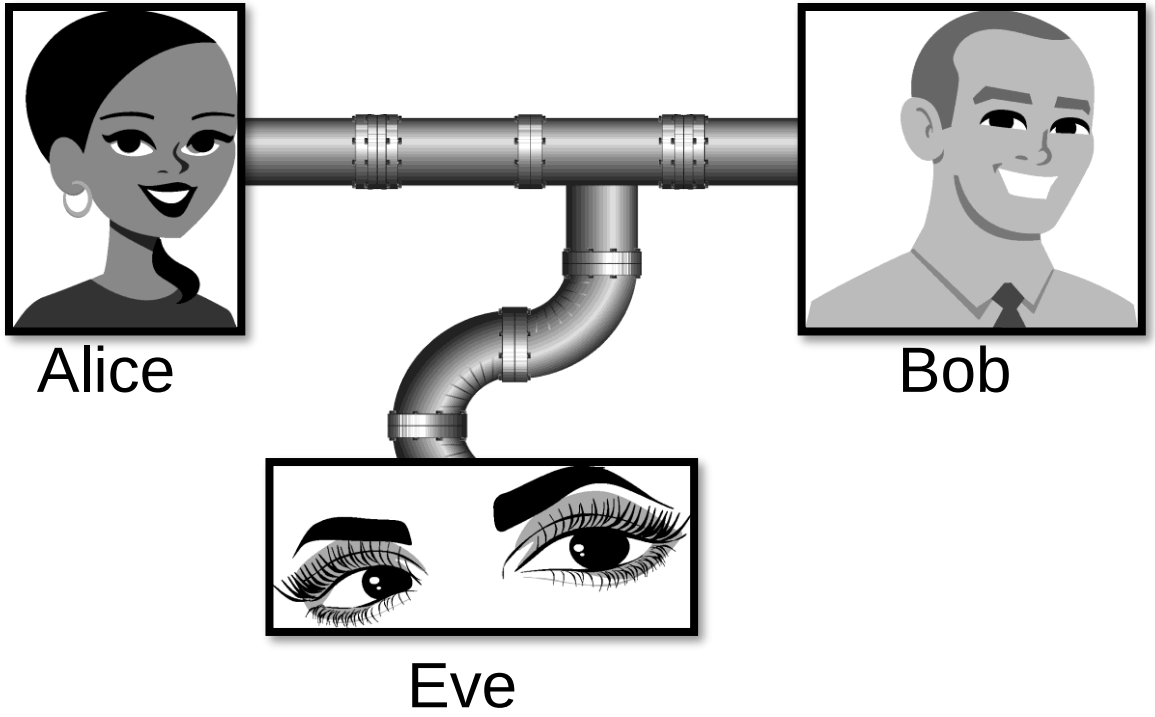
Anonymity

- **Anonymity:** the property that certain records or transactions not to be attributable to any individual.

- **Tools:**
  - **Aggregation:** the combining of data from many individuals so that disclosed sums or averages cannot be tied to any individual.
  - **Mixing:** the intertwining of transactions, information, or communications in a way that cannot be traced to any individual.
  - **Proxies:** trusted agents that are willing to engage in actions for an individual in a way that cannot be traced back to that person.
  - **Pseudonyms:** fictional identities that can fill in for real identities in communications and transactions, but are otherwise known only to a trusted entity.

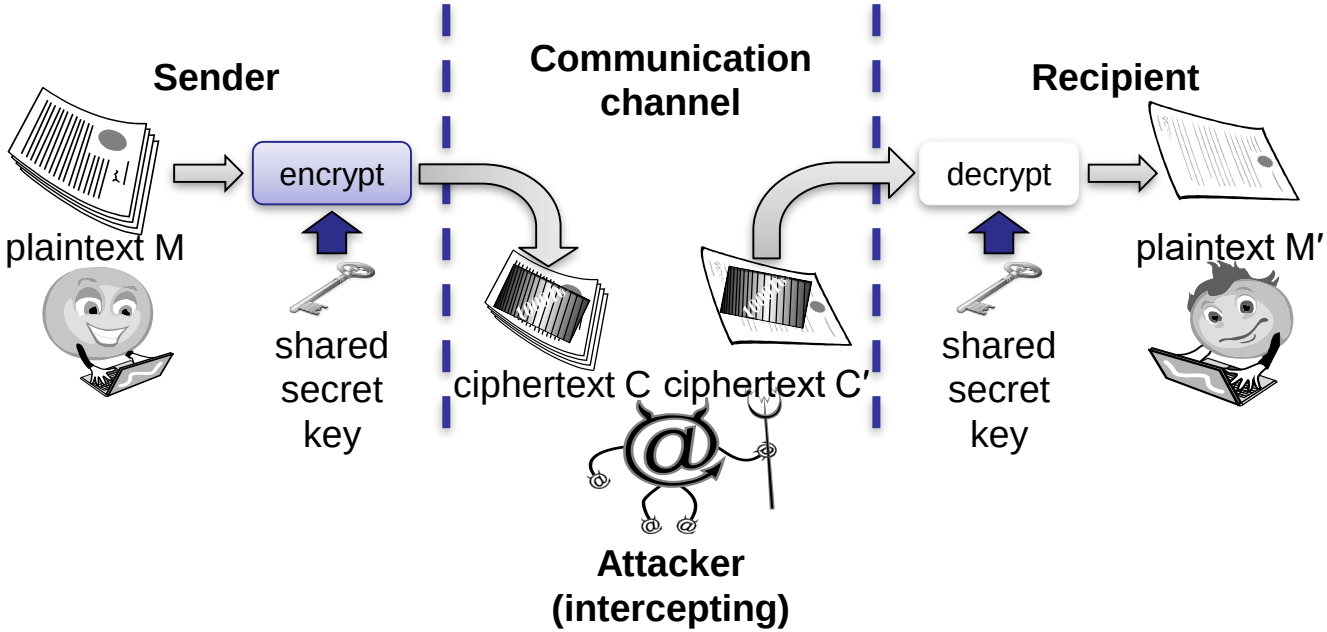
Threats and Attacks

- **Eavesdropping:** the interception of information intended for someone else during its transmission over a communication channel.



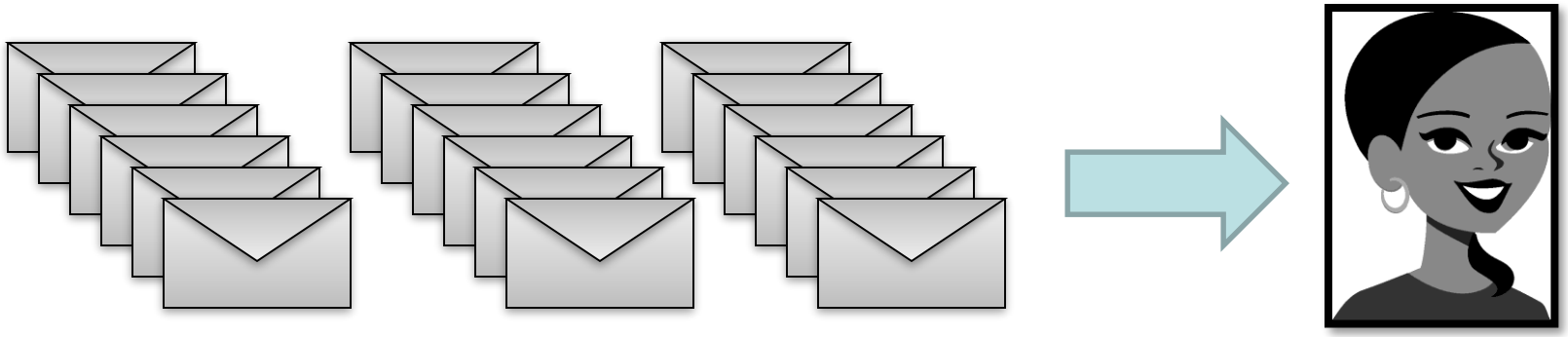
Threats and Attacks

- **Alteration:** unauthorized modification of information.
  - **Example:** the **man-in-the-middle attack,** where a network stream is intercepted, modified, and retransmitted.



Threats and Attacks

- **Denial-of-service:** the interruption or degradation of a data service or information access.
  - **Example:** email **spam,** to the degree that it is meant to simply fill up a mail queue and slow down an email server.



Alice

Threats and Attacks

- **Masquerading:** the fabrication of information that is purported to be from someone who is not actually the author.

"From: Alice"
(really is from Eve)

Threats and Attacks

- **Repudiation:** the denial of a commitment or data receipt.
  - This involves an attempt to back out of a contract or a protocol that requires the different parties to provide receipts acknowledging that data has been received.

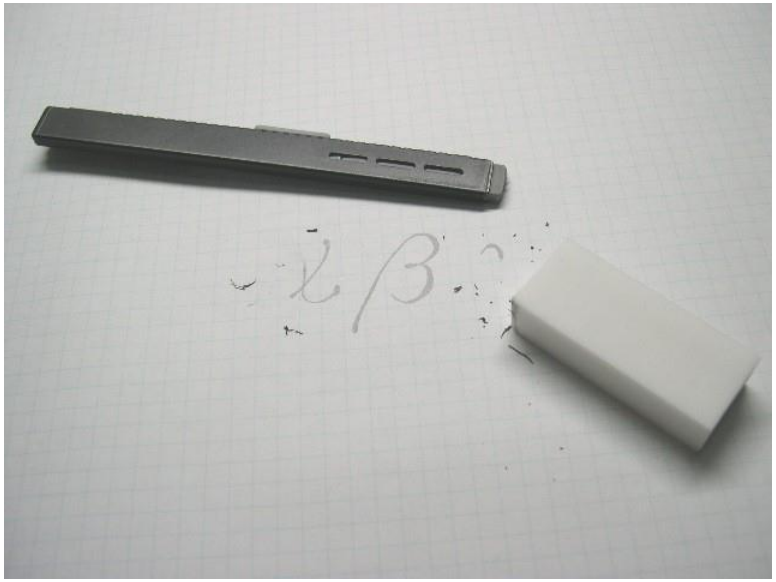
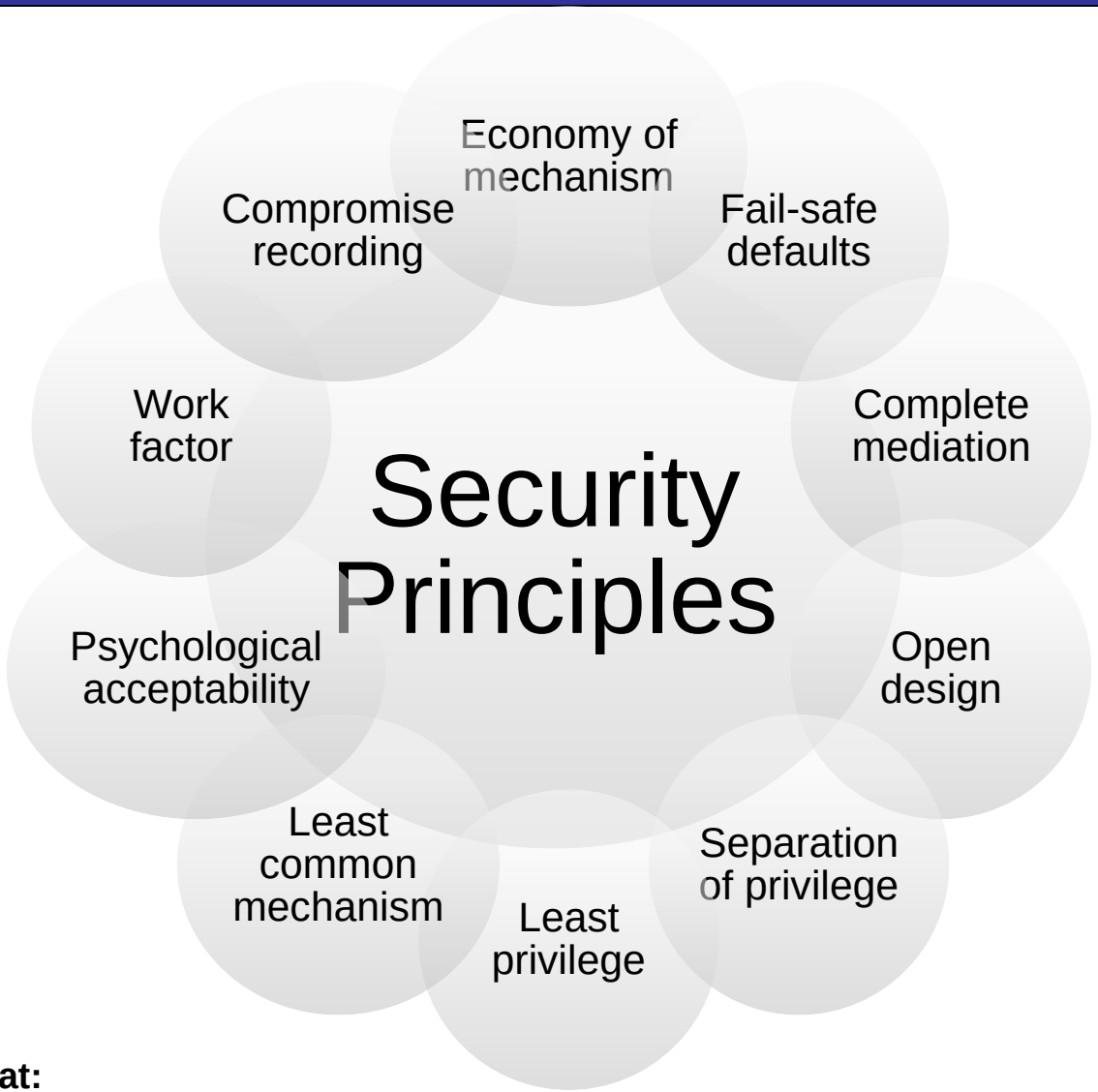


Photo: Wikimedia Commons https://commons.wikimedia.org/wiki/File:Plastic_eraser.jpeg

Threats and Attacks

- **Correlation** and **traceback:** the integration of multiple data sources and information flows to determine the source of a particular data stream or piece of information.

The Ten Security Principles



Read more at:
http://emergentchaos.com/the-security-principles-of-saltzer-and-schroeder
http://web.mit.edu/Saltzer/www/publications/protection/

Economy of mechanism

- This principle stresses **simplicity** in the **design** and **implementation** of security measures.
  - While applicable to most engineering endeavors, the notion of simplicity is especially important in the security domain, since a simple security framework facilitates its understanding by developers and users and enables the efficient development and verification of enforcement methods for it.

Fail-safe defaults

- This principle states that the default configuration of a system should have a **conservative protection scheme**.
  - For example, when adding a new user to an operating system, the default group of the user should have minimal access rights to files and services.
    - Unfortunately, operating systems and applications often have default options that favor usability over security.
    - This has been historically the case for a number of popular applications, such as web browsers that allow the execution of code downloaded from the web server.

Complete mediation

- The idea behind this principle is that every access to a resource must be checked for **compliance with a protection scheme**.
  - As a consequence, one should be wary of performance improvement techniques that save the results of previous authorization checks, since permissions can change over time.
  - For example, an online banking web site should require users to sign on again after a certain amount of time, say, 15 minutes, has elapsed.

Open design

- According to this principle, the security architecture and **design** of a system should be made **publicly available**.
  - Security should rely only on keeping cryptographic keys secret.
  - Open design allows for a system to be scrutinized by multiple parties, which leads to the early discovery and correction of security vulnerabilities caused by design errors.
  - The open design principle is the opposite of the approach known as **security by obscurity,** which tries to achieve security by keeping cryptographic algorithms secret and which has been historically used without success by several organizations.

Separation of privilege

- Separation of the various components of a system, to limit the damage caused by a security breach of any individual component
  - access to different components of a system requires different privileges, so that if an individual component is compromised the damage is limited to that component only.
  - Example: Company checks over $75,000 to be signed by two officers.
  - Don’t put all your eggs in one basket!

Least privilege

- Each program and user of a computer system should operate with the bare **minimum privileges necessary** to function properly.
  - If this principle is enforced, abuse of privileges is restricted, and the damage caused by the compromise of a particular application or user account is minimized.
  - The military concept of **need-to-know** information is an example of this principle.

Least common mechanism

- In systems with multiple users, mechanisms allowing resources to be **shared by more than one user should be minimized**.
  - For example, if a file or application needs to be accessed by more than one user, then these users should have separate channels by which to access these resources, to prevent unforeseen consequences that could cause security problems.
  - Shared resources need resource isolation to prevent becoming a denial-of-service target
  - Restrictive because it limits sharing

Psychological acceptability

- This principle states that user interfaces should be **well designed and intuitive**, and all security-related settings should adhere to what an ordinary user might expect.

Work factor

- According to this principle, the **cost of circumventing** a security mechanism should be compared with the resources of an attacker when designing a security scheme.
  - A system developed to protect student grades in a university database, which may be attacked by snoopers or students trying to change their grades, probably needs less sophisticated security measures than a system built to protect military secrets, which may be attacked by government intelligence organizations.

Compromise recording

- This principle states that sometimes it is more desirable to **record the details** of an intrusion than to adopt more sophisticated measures to prevent it.
  - Internet-connected surveillance cameras are a typical example of an effective compromise record system that can be deployed to protect a building in lieu of reinforcing doors and windows.
  - The servers in an office network may maintain **logs** for all accesses to files, all emails sent and received, and all web browsing sessions.

Topic: Access Control

- Users and groups
- Authentication
- Passwords
- File protection
- Access control lists

- Which users can read/write which files?
- Are my files really safe?
- What does it mean to be root?
- What do we really want to control?

Access Control Matrices

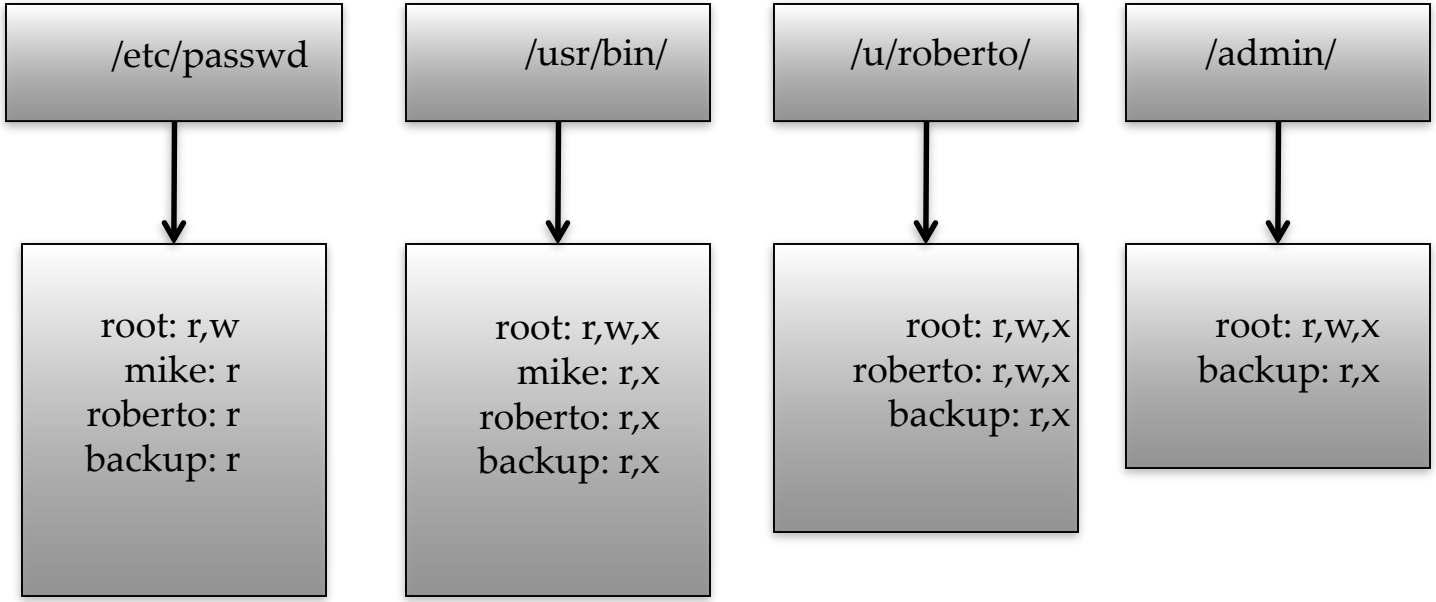
- **A table that defines permissions**.
  - Each row of this table is associated with a **subject,** which is a user, group, or system that can perform actions.
  - Each column of the table is associated with an **object,** which is a file, directory, document, device, resource, or any other entity for which we want to define access rights.
  - Each cell of the table is then filled with the access rights for the associated combination of subject and object.
  - Access rights can include actions such as reading, writing, copying, executing, deleting, and annotating.
  - An empty cell means that no access rights are granted.

Example Access Control Matrix

	/etc/passwd	/usr/bin/	/u/roberto/	/admin/
root	read, write	read, write, exec	read, write, exec	read, write, exec
mike	read	read, exec		
roberto	read	read, exec	read, write, exec	
backup	read	read, exec	read, exec	read, exec
...	...	...	...	...

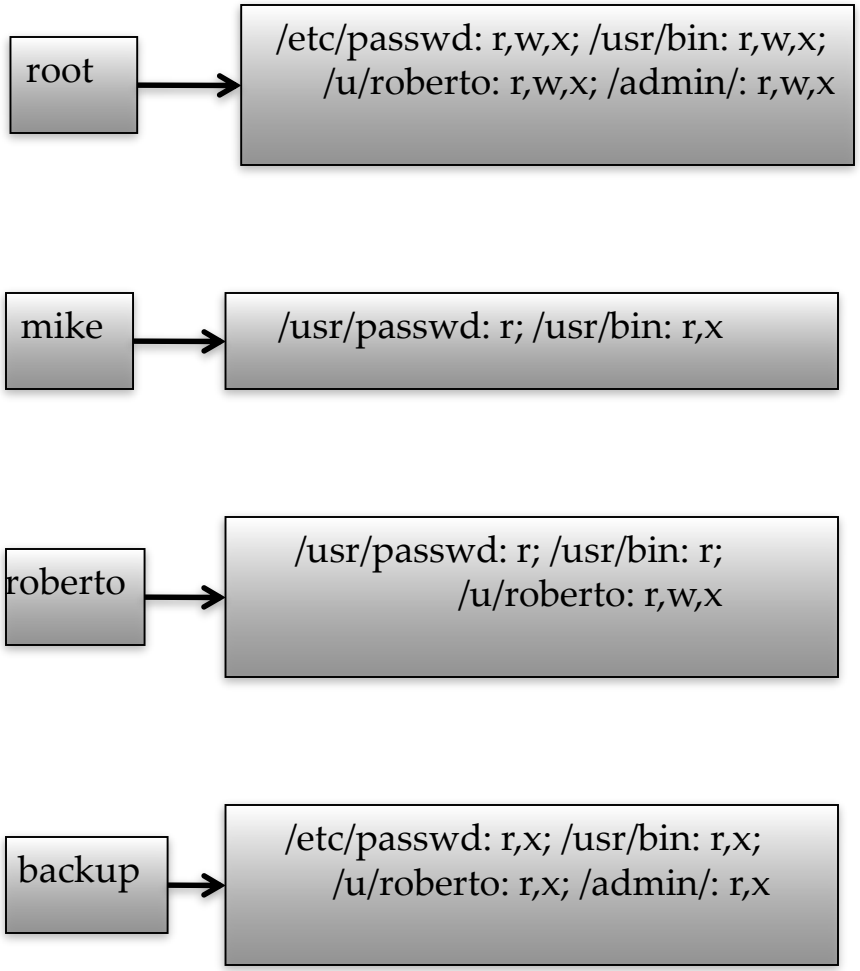
Access Control Lists

- It defines, for each object, o, a list, L, called o's access control list, which enumerates all the subjects that have access rights for o and, for each such subject, s, gives the access rights that s has for object o.



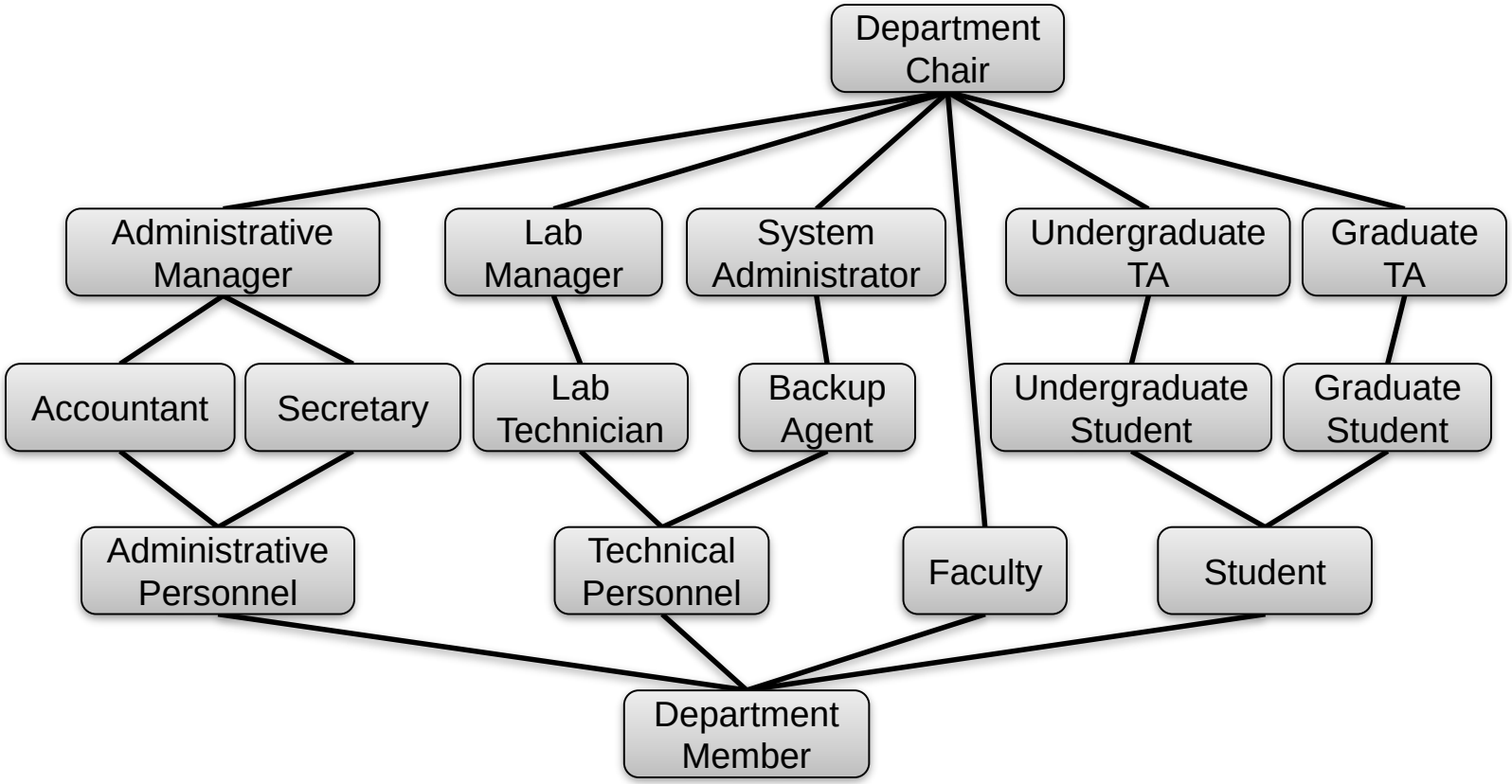
Capabilities

- Takes a subject-centered approach to access control. It defines, for each subject s, the list of the objects for which s has nonempty access control rights, together with the specific rights for each such object.



Role-based Access Control

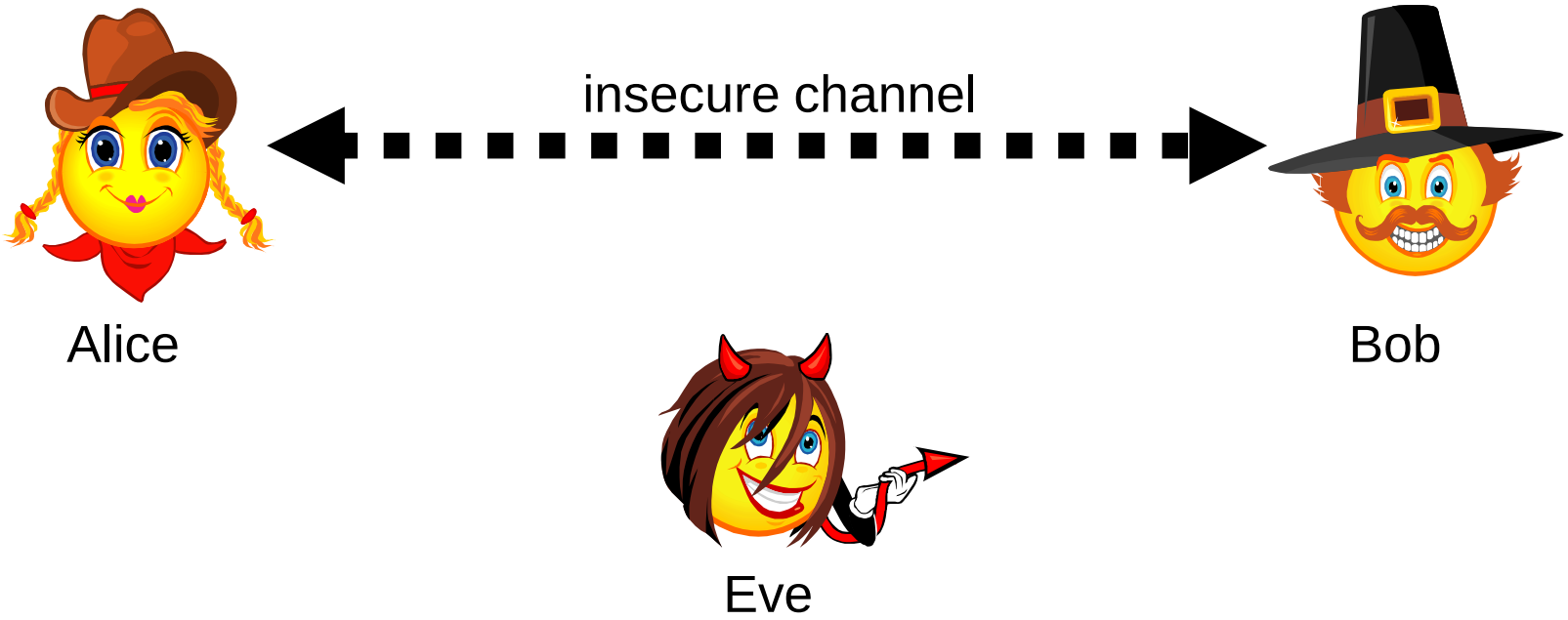
- Define **roles** and then specify access control rights for these roles, rather than for subjects directly.



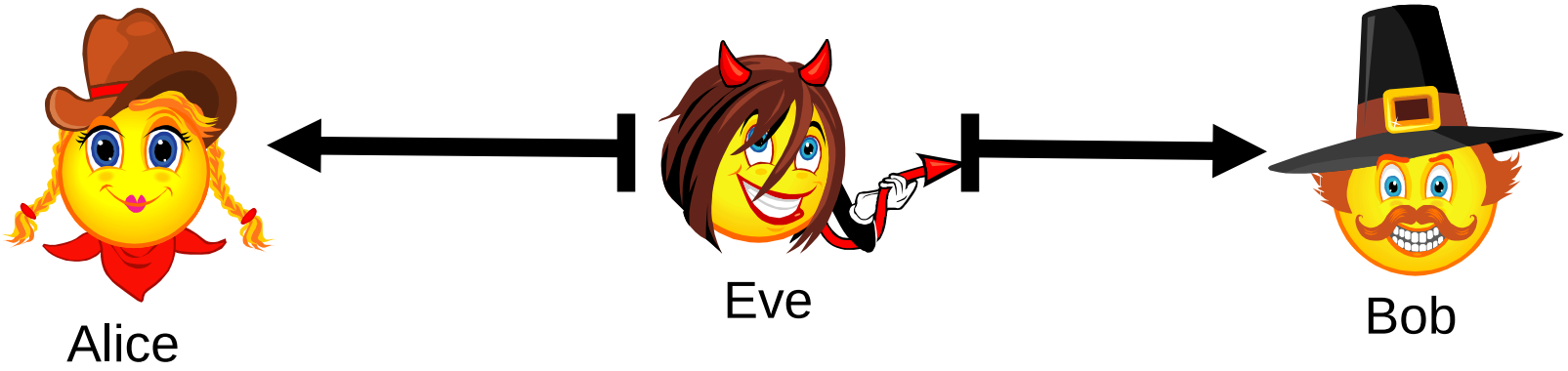
Crypto Crash Course

Attacks against communication

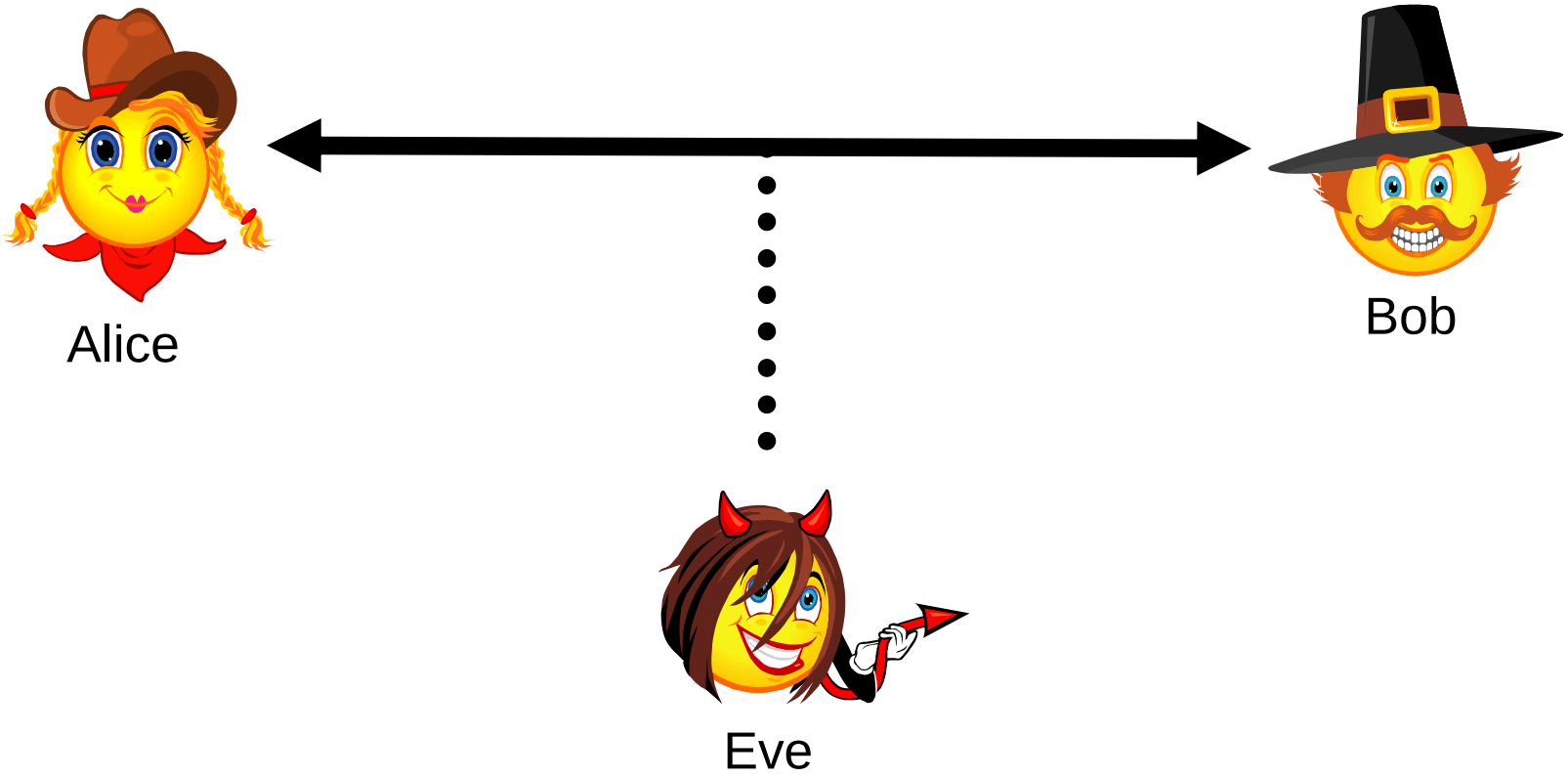
- Passive attacks
- Active attacks



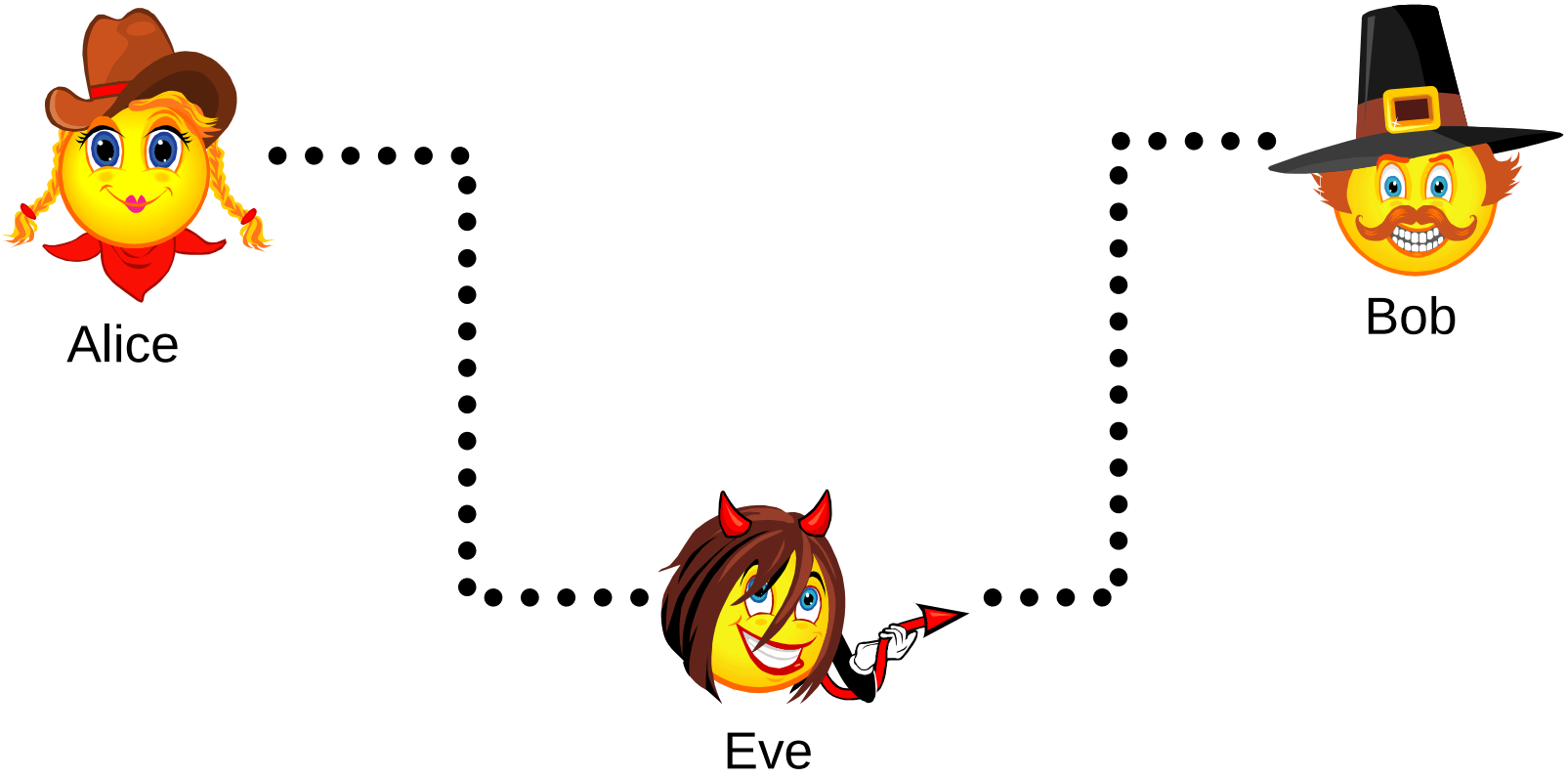
Attacks - interruption



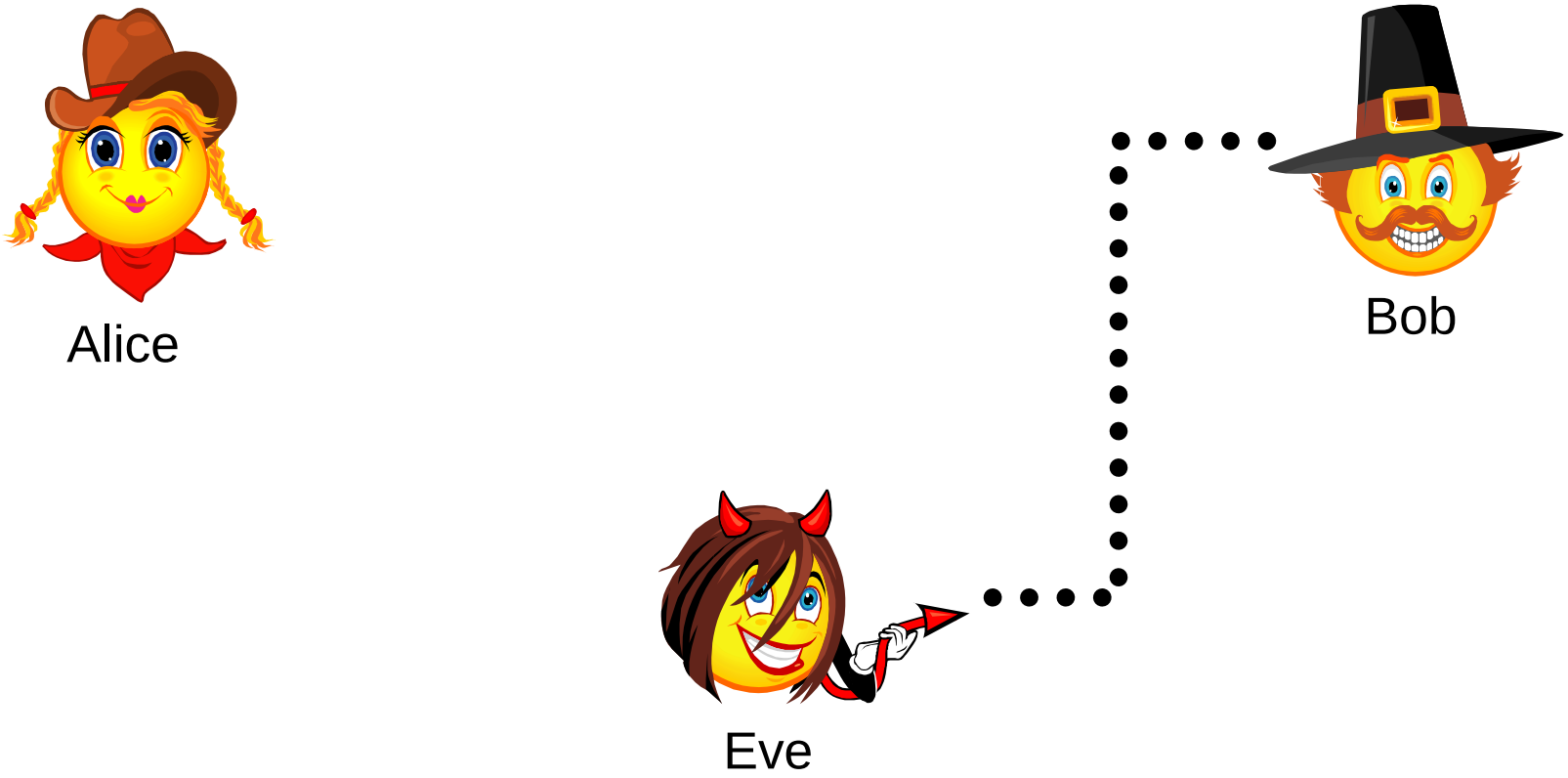
Attacks - interception



Attacks - modification



Attacks - injection



Security mechanisms

- Cryptography
  - Encryption
  - Digital Signatures
  - Message authentication codes
  - …
- Network security (firewall, IDS etc.)
- Hardware/Software protection
- Ad-hoc techniques
- …

Cryptology

- Cryptology
  - The study of cryptography and cryptanalysis
- Cryptography
  - The study of mathematical techniques related to aspects of information security
- Cryptanalysis
  - The study of mathematical techniques for attempting to defeat information security services

Security goals for data communication

- Confidentiality
  - Ensures secrecy of the message
  - Achieved using encryption
- Integrity
  - Ensures data has not been altered in an unauthorized manner since the time it was transmitted
  - Achieved using MAC (message authentication code) or dig. signature
- Authentication
  - Ensures message really comes from the stated sender
  - Achieved using MAC (message authentication code) or dig. signature
- Non-repudiation
  - Ensures a party cannot deny the validity of the messages it creates
  - Achieved using digital signature

Randomness

- Is essential for cryptography (e.g., for generating keys)
- Random numbers: a sequence of numbers x_1, x_2, … , s.t. for any integer k>0, it is **impossible** for an observer to predict x_k even if all of x_1, x_2, …, x_{k-1} are known
- True randomness needs an unpredictable source
  - Example: atmospheric noise, radioactive decay etc.
  - Example: hardware RNG (based on thermal noise, photoelectric effect)
  - Computers cannot generate true randomness

Pseudo-randomness

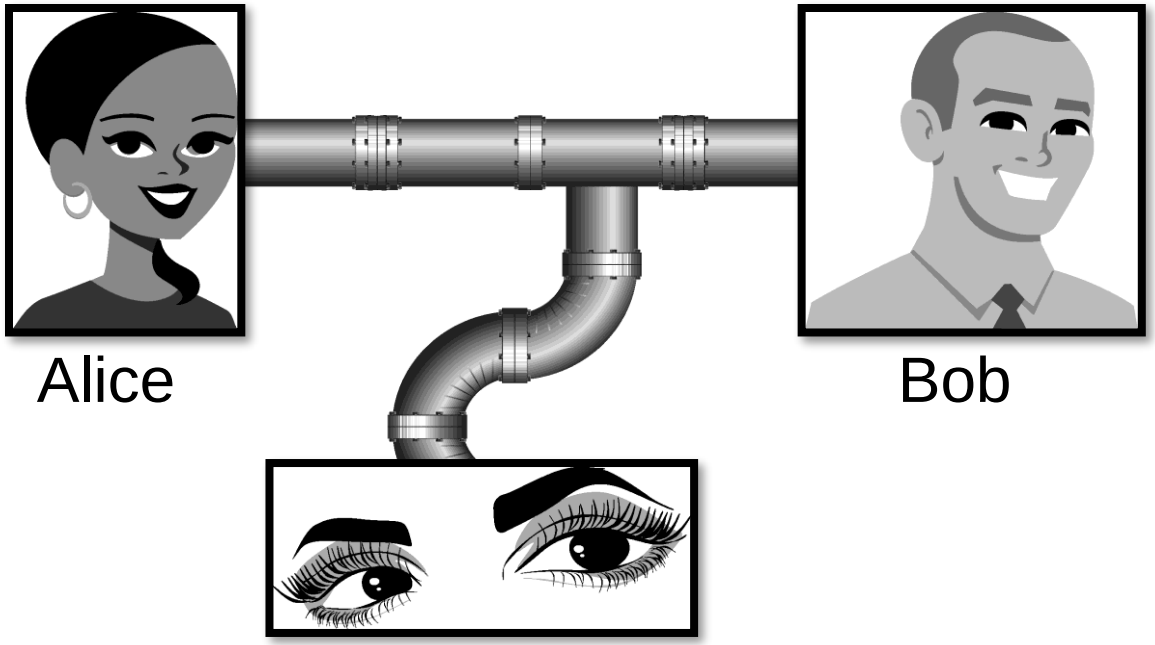
- Pseudo-random numbers: a sequence of numbers x_1, x_2, … , s.t. for any integer k>0, it is **hard** for an observer to predict x_k even if all of x_1, x_2, …, x_{k-1} are known
- Hard means computationally hard for all algorithms that run in polynomial time
- Pseudo-random number generator (PRNG) on computers
  - User keystrokes, I/O, least-significant digit voltage measurements
  - Pool of random numbers is constantly replenished

Cryptographic hash function

- Takes an input of any size and returns a fixed-size string
- Informal requirements
  - One-way (non-invertible)
  - Produces different outputs for different inputs (with high likelihood)
- Formally:
  - *First pre-image resistance*: given h(x), cannot find x
  - *Second pre-image resistance*: given x and h(x), cannot find $y \neq x$ s.t $h(y) = h(x)$
  - *Collision resistance*: cannot find any pair x,y, with $x \neq y$ s.t. $h(x) = h(y)$
- It is an unkeyed cryptographic primitive (publicly computable, no secret involved)
- Examples: SHA-1 (160 bits output), SHA-256 (256 bits output)

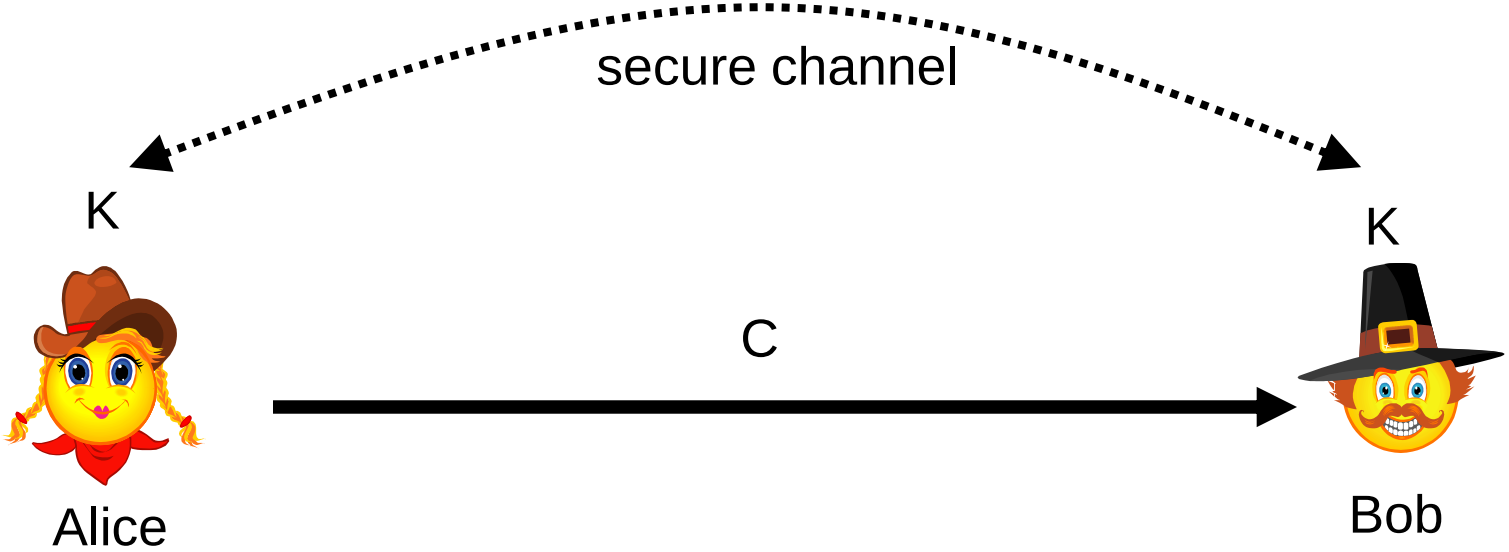
Cryptographic Concepts

- **Encryption**: a means to allow two parties, customarily called Alice and Bob, to establish confidential communication over an insecure channel that is subject to eavesdropping.



Symmetric key encryption

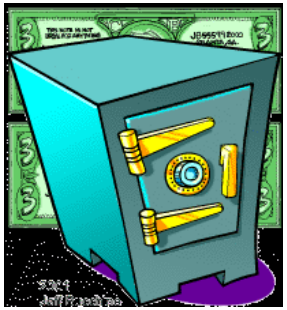
- Alice and Bob have a trust relationship (i.e., they share a secret key K)



generate ciphertext
$C = E_K(M)$

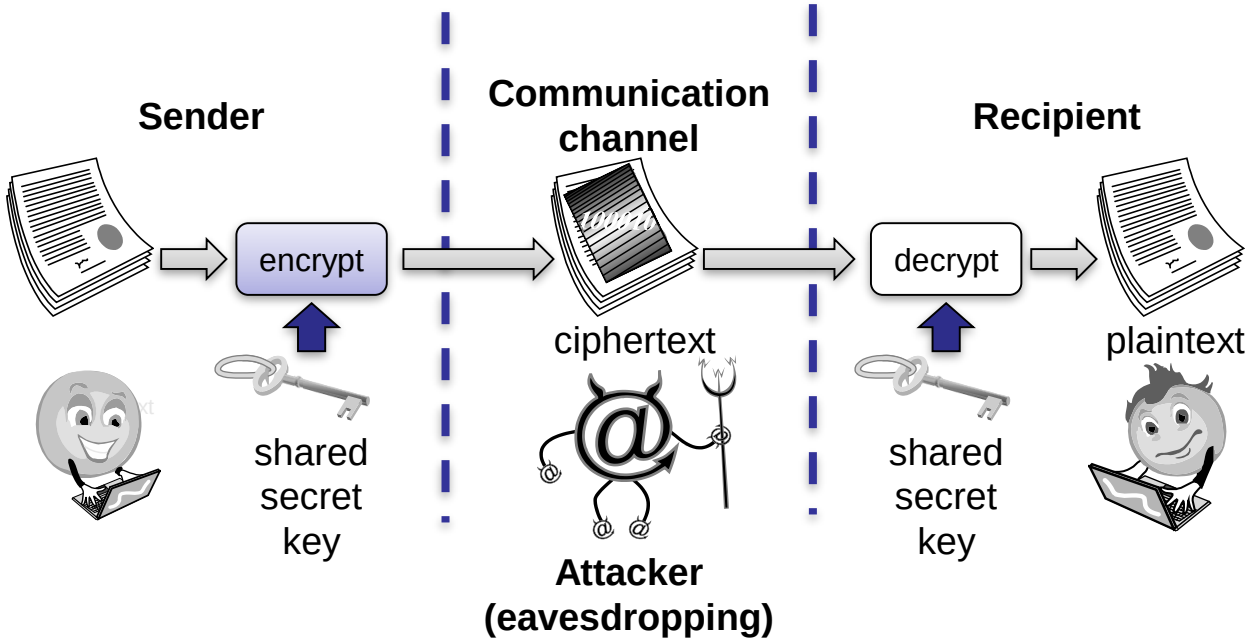
retrieve plaintext
$M = D_K(C)$

- Analogy with a safe



Encryption and Decryption

- The message M is called the **plaintext.**
- Alice will convert plaintext M to an encrypted form using an encryption algorithm E that outputs a **ciphertext C for M.**



Encryption and Decryption

- As equations:

$$C = E(M)$$

$$M = D(C)$$

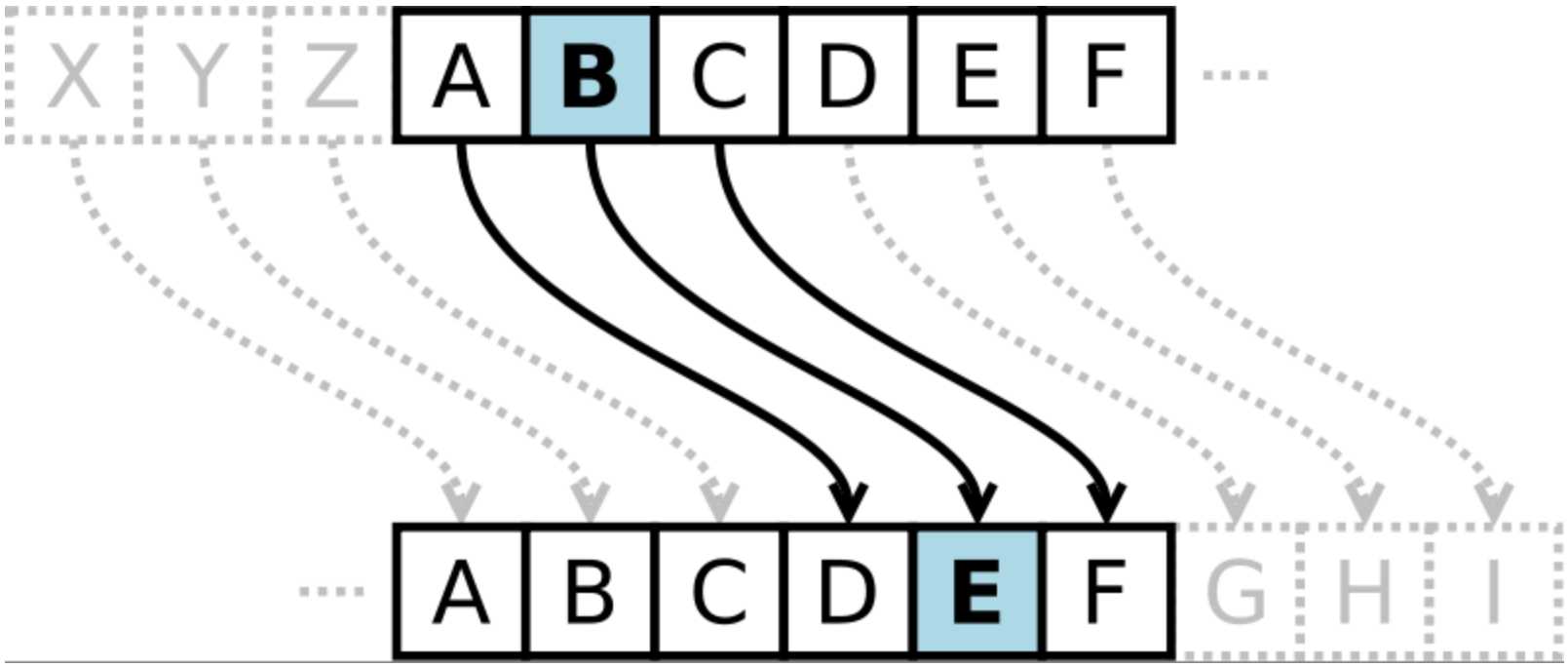
- The encryption and decryption algorithms are chosen so that it is infeasible for someone other than Alice and Bob to determine plaintext M from ciphertext C. Thus, ciphertext C can be transmitted over an insecure channel that can be eavesdropped by an adversary.

Cryptosystem

1. The set of possible plaintexts
2. The set of possible ciphertexts
3. The set of encryption keys
4. The set of decryption keys
5. The correspondence between encryption keys and decryption keys
6. The encryption algorithm to use
7. The decryption algorithm to use

Caesar Cipher

- Replace each letter with the one "three over" in the alphabet.
  - Note that modern ciphers are much more complex



Public domain image from http://commons.wikimedia.org/wiki/File:Caesar3.svg

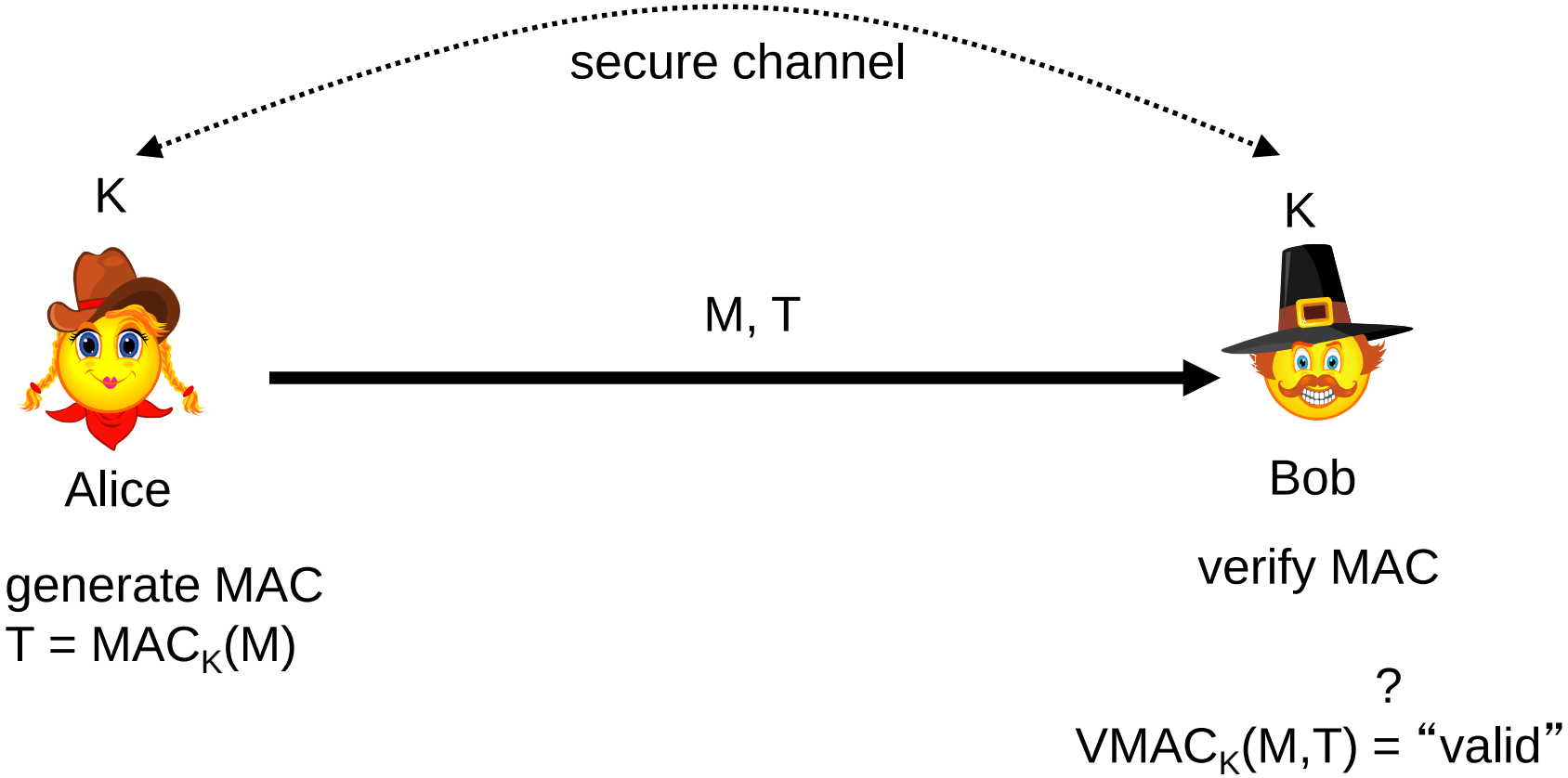
Symmetric key encryption

- A symmetric key encryption scheme is a collection of three algorithms (G, E, D)
  - Key generation algorithm G generates a key K
  - Encryption algorithm E: $C = E_K(M)$
  - Decryption algorithm D: $M = D_K(C)$
- The following should always hold true:
  - $D_K(E_K(M)) = M$, for all K and M

- Provides: confidentiality
- Does not provide: authentication, integrity, non-repudiation
- It is a keyed cryptographic primitive
- Example: AES, DES, DES3, Twofish, Serpent etc.

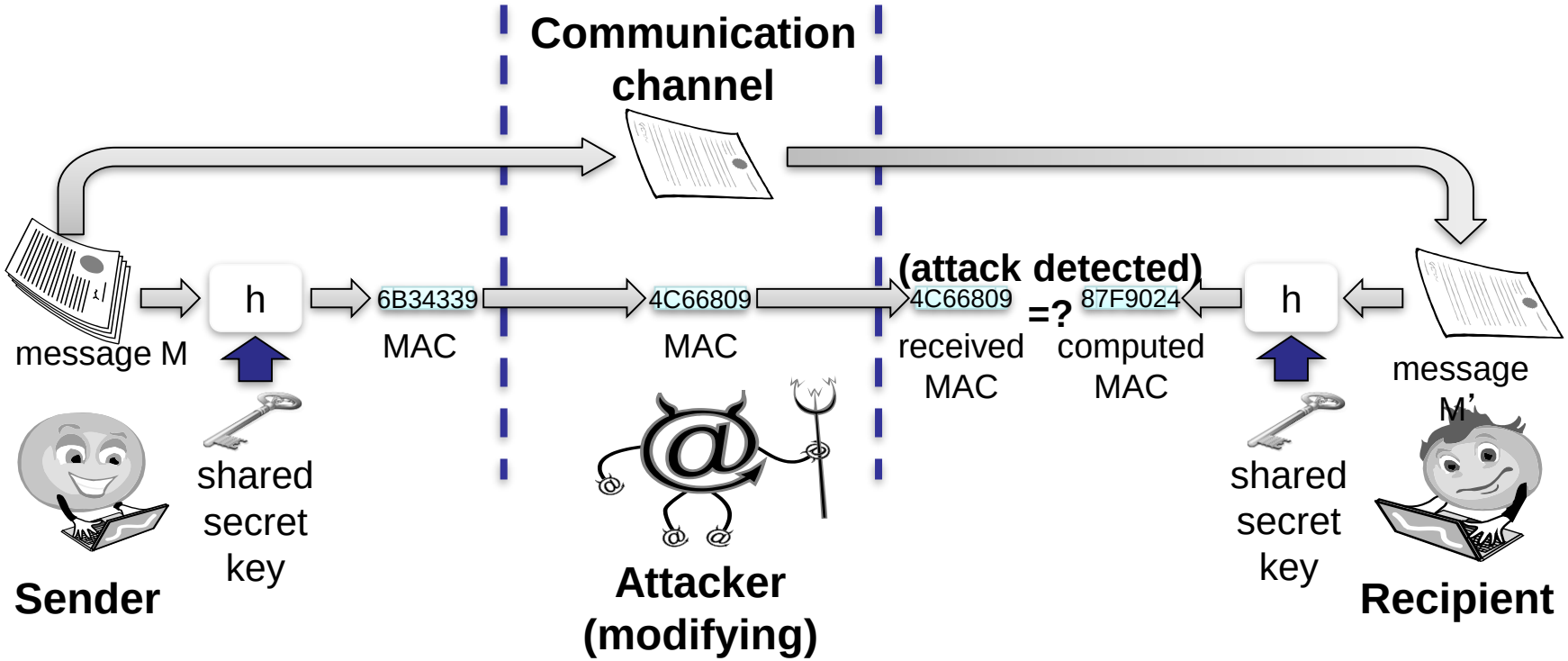
Message Authentication Code (MAC)

- Alice and Bob have a trust relationship (i.e., they share a secret key K)



Message Authentication Codes

- Allows for Alice and Bob to have data integrity, if they share a secret key in advance.
- Given a message M, Alice computes H(K||M) and sends M and this hash to Bob.



Message authentication code

- A message authentication code is a collection of three algorithms (G, MAC, VMAC)
  - Key generation algorithm G: generates a key K
  - Authentication tag generation algorithm MAC: $T = MAC_K(M)$
  - Authentication tag verification algorithm VMAC:
    "result" = $VMAC_K(M,T)$, where "result" is "valid" or "invalid"
- The following should always hold true:
  - $VMAC_K(M,T)$ = "valid", if $T = MAC_K(M)$
    = "invalid", otherwise

- Provides: authentication, integrity
- Does not provide: confidentiality, non-repudiation
- It is a keyed cryptographic primitive
- Example: HMAC

Public key cryptography

- Came into existence in 1976
- Also known as asymmetric cryptography
- Analogy with a mailbox

- Is possible because of one-way trapdoor functions
  - Example: $f(x) = x^3 \bmod n$, $n = pq$ (based on the integer factorization problem)

Public key signature

Each entity has:

- a public key (Pub), which is made public
- a private key (Priv), which is kept secret

$(Pub_A, Priv_A)$

Alice

M, Sig

Bob

generate signature
$Sig = S_{PrivA}(M)$

verify signature

$V_{PubA}(M, Sig) \stackrel{?}{=}$ "valid"

Public key signature

- Entities don't need to establish a secret key or a trust relationship ahead of time
- A public key signature scheme is a collection of three algorithms (G, S, V)
  - Key generation algorithm G: generates a pair of keys (Pub, Priv)
  - Signature generation algorithm S: Sig = $S_{Priv}(M)$
  - Signature verification algorithm V:
    "result" = $V_{Pub}(M,Sig)$, where "result" is "valid" or "invalid"
- The following should always hold true:
  - $V_{Pub}(M, Sig))$ = "valid", if Sig = $S_{Priv}(M)$
    = "invalid", otherwise

- It is infeasible to derive the private key from the public key
- Provides: authentication, integrity, non-repudiation
- Does not provide: confidentiality
- It is a keyed cryptographic primitive
- Example: RSA, DSA

Public key encryption

Each entity has:

- a public key (Pub), which is made public
- a private key (Priv), which is kept secret

$(Pub_B, Priv_B)$

Alice

C

Bob

generate ciphertext
$C = E_{PubB}(M)$

retrieve plaintext
$M = D_{PrivB}(C)$

Public-Key encryption

- Bob has two keys: a **private key,** $Priv_B$, which Bob keeps secret, and a **public key,** Pub_B, which Bob broadcasts widely.
  - In order for Alice to send an encrypted message to Bob, she need only obtain his public key, Pub_B, use that to encrypt her message, M, and send the result, $C = E_{PubB}(M)$, to Bob. Bob then uses his secret key to decrypt the message as $M = D_{PrivB}(C)$.

Public key encryption

- Entities don't need to establish a secret key or a trust relationship ahead of time
- A public key encryption scheme is a collection of three algorithms (G, E, D)
  - Key generation algorithm G: generates a pair of keys (Pub, Priv)
  - Encryption algorithm E: $C = E_{Pub}(M)$
  - Decryption algorithm D: $M = D_{Priv}(C)$
- The following should always hold true:
  - $D_{Priv}(E_{Pub}(M)) = M$

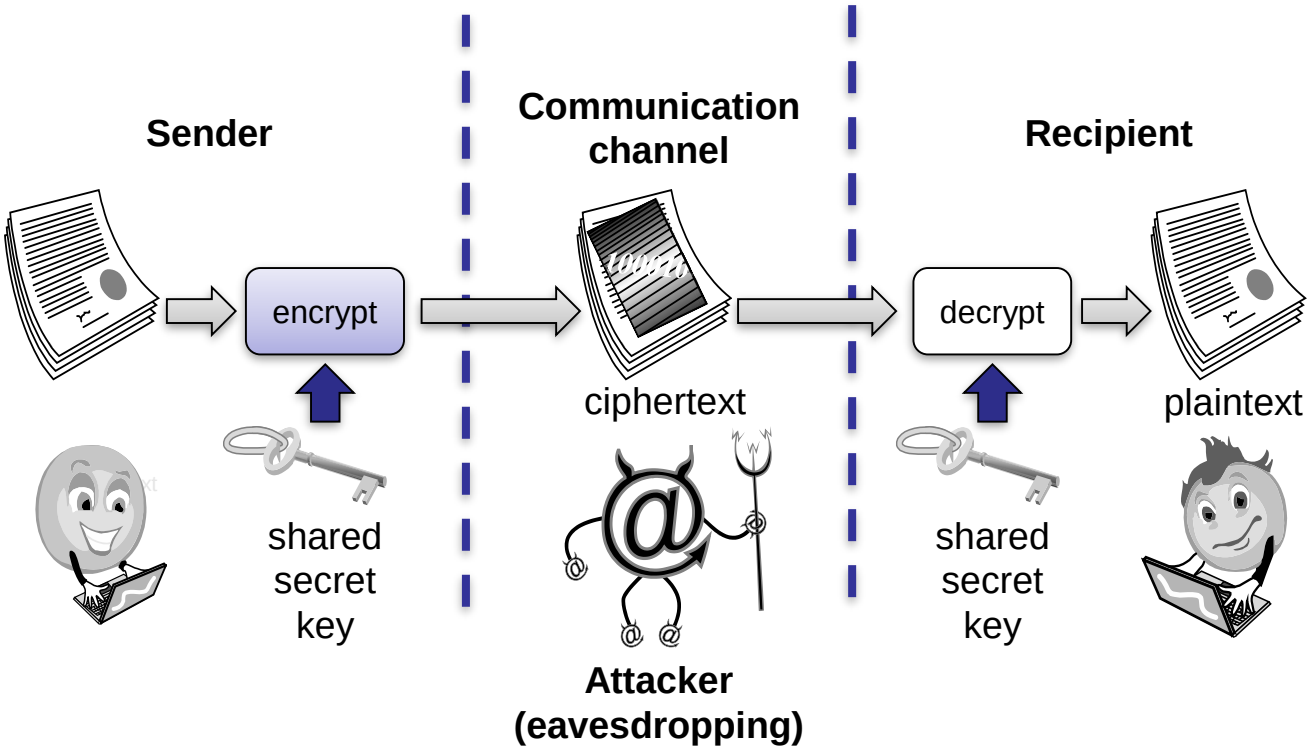
- It is infeasible to derive the private key from the public key
- Provides: confidentiality
- Does not provide: authentication, non-repudiation
- It is a keyed cryptographic primitive
- Example: RSA encryption, El-Gamal encryption

Public key cryptography

- Advantages over symmetric key crypto
  - Key management
    - Key establishment: does not require secure channel to transmit secret keys
    - Key distribution: does not require $O(n^2)$ keys to be managed to communicate with n entities
    - Can provide non-repudiation
- Disadvantages over symmetric key crypto
  - Slower (orders of magnitude)
- Not meant to completely replace symmetric key cryptography, but to supplement it
  - $E_{Pub}(K)$, $E_K(M)$

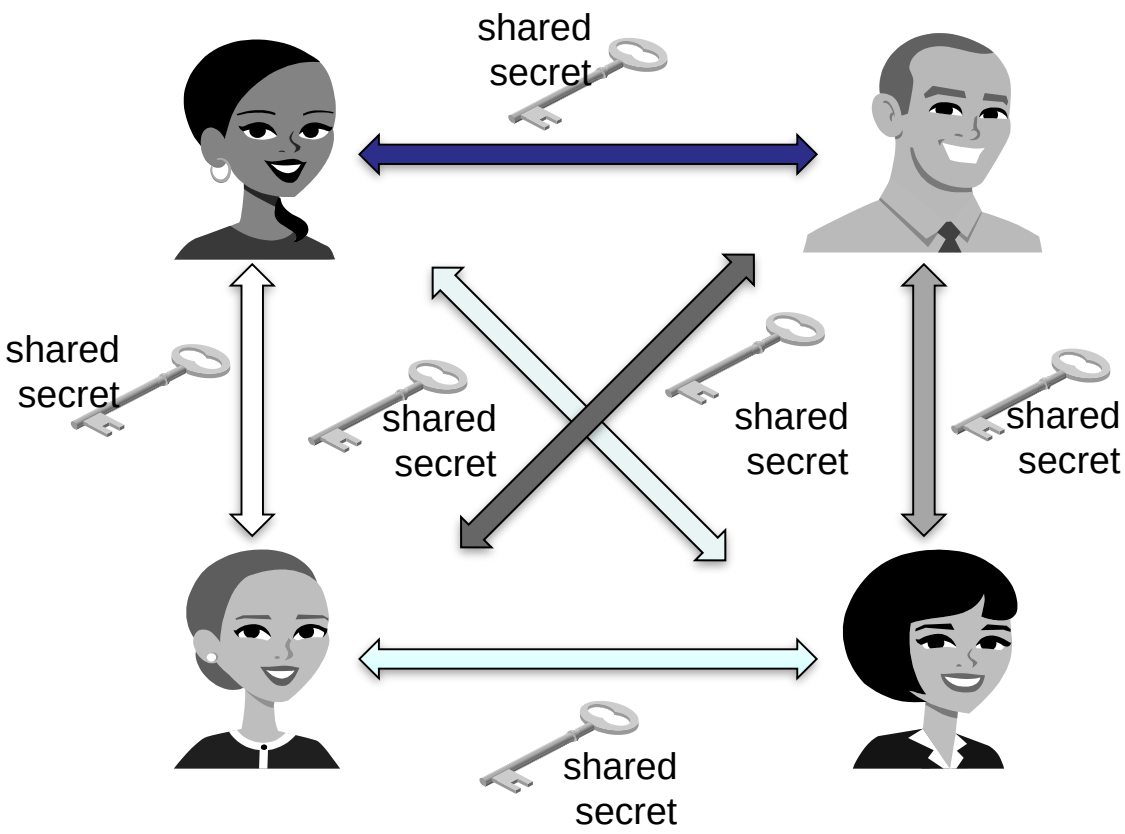
Symmetric Key Cryptosystems

- Alice and Bob share a secret key, which is used for both encryption and decryption.



Symmetric Key Distribution

- Requires each pair of communicating parties to share a (separate) secret key.



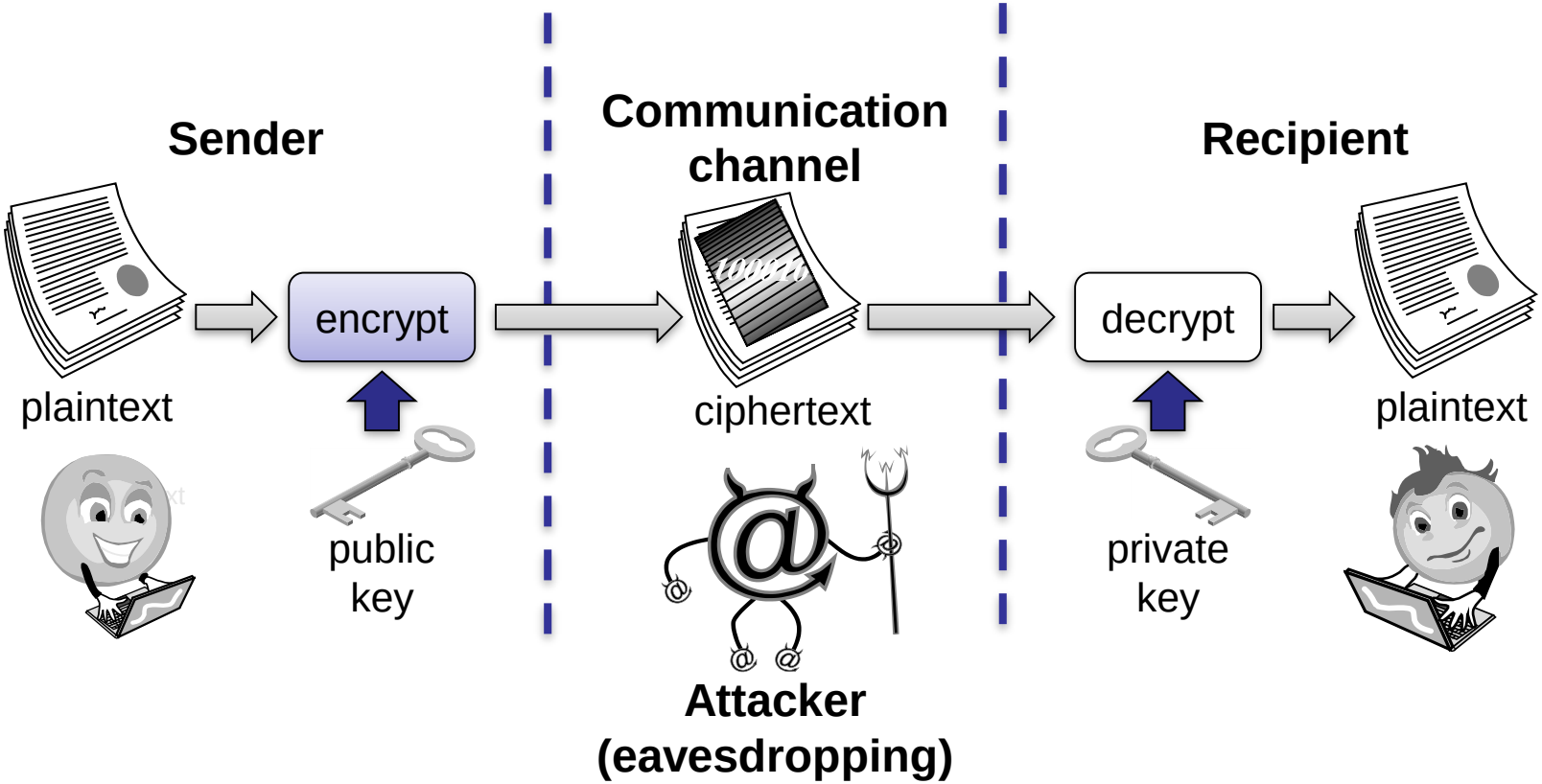
n (n–1)/2 keys

Public-Key Cryptography

- Bob has two keys: a **private key,** SB, which Bob keeps secret, and a **public key,** PB, which Bob broadcasts widely.
  - In order for Alice to send an encrypted message to Bob, she need only obtain his public key, PB, use that to encrypt her message, M, and send the result, $C = E_{PB}(M)$, to Bob. Bob then uses his secret key to decrypt the message as $M = D_{SB}(C)$.

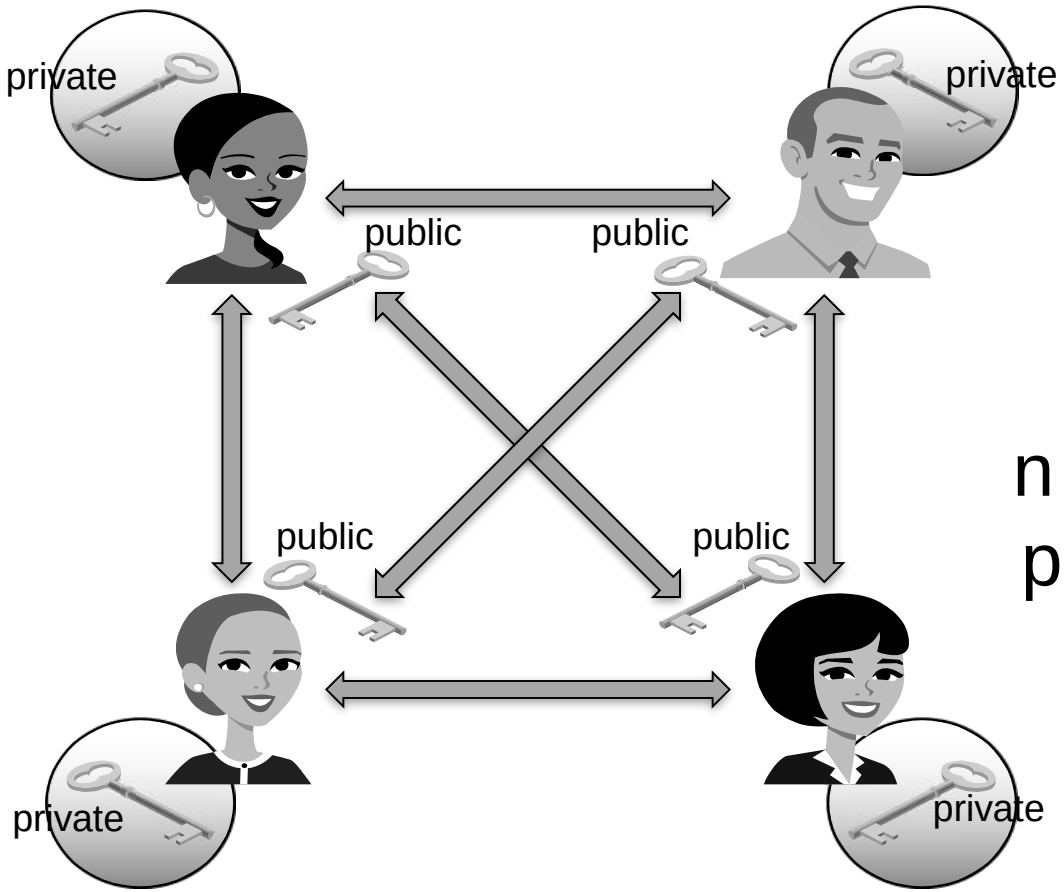
Public-Key Cryptography

- Separate keys are used for encryption and decryption.



Public Key Distribution

- Only one key pair is needed for each recipient



n key pairs

What to read?

- Chapter 1 from the Textbook
- Parts of Chapter 8 for the Cryptographic Concepts